Коллоквиум по C++ ПИ 1 курс весна

github.com/int28t/hse-se-lecture-notes

Содержание

1 Вокруг языка C++ **6**

1.1 Темы . . . 6

1.2 Контрольные вопросы . . . 6

1.2.1 Какие стандарты языка существуют? Каков цикл выпуска новых стандартов? . . . 6

1.2.2 Что такое The Zero Overhead Principle? Привести пример появления этого принципа в коде. . . . 6

1.2.3 Какие основные директивы CMake/файлов проекта? Для чего нужна директива `add_executable`? Каковы её параметры? . . . 7

1.2.4 Что такое интерфейс и что такое реализация? Что из них идеальное, что материальное? . . . 8

1.2.5 Примеры разделения на интерфейс и реализацию в C++ . . . 8

1.2.6 Для чего используются заголовочные файлы в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними нельзя? Какие расширения встречаются? Какова связь между заголовочными файлами и принципом интерфейс–реализация? . . . 9

1.2.7 Для чего используются модули трансляции в C++? Что в них обычно помещают? Что с ними делают и чего нельзя? Какие расширения встречаются? Связь с интерфейс-реализацией? . . . 9

1.3 Примеры и заметки . . . 10

1.3.1 Header guard (классическая реализация) . . . 10

1.3.2 Пример с `pragma once` . . . 11

1.3.3 Короткие практические рекомендации . . . 11

2 Основы C++ **12**

2.1 Темы . . . 12

2.2 Контрольные вопросы и развёрнутые ответы . . . 12

2.2.1 Каково назначение препроцессора языка C(++)? . . . 12

2.2.2 С какими сущностями манипулирует препроцессор? Имеет ли для него принципиальное значение синтаксис и семантика языка C++? . 13

2.2.3 На примере директивы `assert` покажите, как препроцессор меняет код программы для конфигураций сборки `debug` и `release`. . . . 13

2.2.4 Что содержит и что не содержит листинг (результат работы препроцессора)? . . . 13

2.2.5 Приведите пример конфигурирования текста исходного кода программы условными директивами препроцессора (напр., включения фрагмента кода, релевантного определённой ОС). . . . 14

2.2.6 Как включить в модуль системный заголовочный файл? — и заголовочный файл из проекта? . . . 14

2.2.7 В чем заключается проблема повторного (косвенного) включения заголовочного файла? Приведите пример. 14
2.2.8 Что такое header guard и какую проблему он призван предотвратить? Приведите конкретный пример. 15
2.2.9 Как правильно именовать header guard (стилевые договорённости)? . 15
2.2.10 Реализация с помощью `#pragma once` 15
2.2.11 Можно ли использовать заголовочный файл без header guard и если да, то какие ограничения это накладывает? 16
2.2.12 Что обычно кладут в заголовочный файл — интерфейсную часть некоторого модуля `foo`? А что обычно не кладут — и почему? 16
2.2.13 Чем в C++ описание (declaration) принципиально отличается от определения (definition)? Как можно наблюдать эффект описания и эффект определения? 16
2.3 Короткие примеры (безопасные для вставки в `.tex`) 17
2.4 Итог и рекомендации для экзамена / коллоквиума 17

3 Базовые конструкции C++ **18**
3.1 Темы 18
3.2 Контрольные вопросы и развернутые ответы 18
3.2.1 Какие ключевые слова введены/важны (особенно с C++11 и новее) и каково их назначение? 18
3.2.2 Какова базовая структура программы на C++? Какие основные компоненты и на каком уровне видимости встречаются? Какие из них обрабатываются какими инструментами построения программы? . . . 19
3.2.3 Какие идентификаторы корректны для объектов, типов, функций, глобальных констант, символов препроцессора? Как влияют кодстайл договорённости? 20
3.2.4 Что такое инструкция (statement)? Какой признак указывает на появление инструкции? Примеры исключений. 20
3.2.5 Что такое инструкция описания объекта? Пример. 20
3.2.6 Что такое инструкция описания функции? Пример. 20
3.2.7 Что такое инструкция определения и описания объекта? Пример. . . 20
3.2.8 Может ли существовать только описание без определения и наоборот? Примеры. 21
3.2.9 Как производится инициализация объектов и когда? Варианты и различия. 21
3.2.10 Что такое унифицированный (list-)инициализатор? Когда рекомендуется применять? 21
3.2.11 Пример инструкции определения и описания с инициализацией. Когда это важно? 21
3.2.12 Блок инструкций: зачем и можно ли использовать блок без заголовка? 22
3.2.13 Как блок влияет на область видимости? 22
3.2.14 Как в C++ определить именованный блок инструкций (функцию)? . 22
3.2.15 Объекты и типы: связь, характеристики объектов и типов. 22
3.2.16 Может ли существовать объект без типа? 22
3.2.17 Как объекты представляются в оперативной памяти? Какие характеристики влияют? 22
3.2.18 Оператор `sizeof`: применение и возвращаемый тип 23
3.2.19 Что такое операционная семантика типа? ADT примеры 23
3.2.20 Классификация типов в C++ (трёхтаксонная и двухтаксонная) . . . 23

3.2.21 Модификаторы типов для производных типов . . . 23
3.2.22 Целочисленные типы: какие и как определяется знаковость, диапазоны 23
3.2.23 Типы с плавающей точкой: структура и специальные значения . . . 23
3.2.24 Инициализация числовых объектов, литералы, narrowing . . . 23
3.2.25 Что такое POD-тип? Примеры и особенности . . . 24
3.2.26 Что такое выражение, оператор, операнд? Может ли выражение состоять из одного операнда/оператора? . . . 24
3.2.27 Классификация операторов по арности и приоритетности . . . 24
3.2.28 Охарактеризуйте оператор (на примере): возвращаемое значение и случаи отсутствия возвращаемого значения . . . 24
3.2.29 Постройте дерево разбора для выражения (пример) . . . 24
3.2.30 Операция присваивания. Различие между инициализацией и присваиванием . . . 25
3.2.31 Что такое инструкция-выражение? . . . 25
3.2.32 Как работает `if` (короткий, длинный, с инициализацией)? . . . 25
3.2.33 Как работает `switch`? Зачем `break` и когда его не использовать? . . . 25
3.2.34 Соотношение тернарного оператора `?:` и `if` . . . 25
3.2.35 Цикл `while`: назначение и особенности . . . 26
3.2.36 Цикл `for` и range-based `for` . . . 26
3.2.37 Цикл `do..while`: особенности . . . 26
3.2.38 Может ли описание существовать без определения и наоборот? . . . 26
3.2.39 Как с помощью блока преодолеть ограничение одной инструкции, например в теле `if`? . . . 26
3.2.40 Разница между `short` и `int` кроме размера? . . . 27
3.2.41 Разница между `char` и `int` кроме размера? . . . 27

4 Потоки и строки STL **28**
4.1 Темы . . . 28
4.2 Контрольные вопросы и ответы . . . 28
4.2.1 Что подразумевают под моделью ввода-вывода консольного приложения (CLI-application)? Какие основные компоненты ввода-вывода задействуются для него? . . . 28
4.2.2 Что такое стандартный поток ввода и стандартный поток вывода? Какими объектами они представлены? Какая область видимости этих объектов? . . . 29
4.2.3 Какой заголовочный файл необходим для доступа к стандартным потокам? Где описаны и где определены эти объекты? . . . 29
4.2.4 Какие типы данных у стандартных потоков и какая иерархия наследования? . . . 29
4.2.5 Какие операции ввода/вывода используются (операторы » и «)? Можно ли их перепутать? Как они работают с пробелами? Что возвращают эти операторы? . . . 30
4.2.6 Какие функции ввода/вывода можно использовать наравне с операторами? Как прочитать целиком строку с пробелами? . . . 30
4.2.7 Что такое состояние потока? Какие флаги идентифицируют состояние (`good`, `eof`, `fail`, `bad`)? Примеры «сломать» поток поправимо и непоправимо; как сбросить флаги? . . . 30
4.2.8 Что такое неявное приведение потока к булевому типу? Как работают проверки `if (s)` и `if (!s)`? . . . 31

4.2.9 Идиоматические конструкции: чтение файла строка за строкой и чтение слов (токенов) . . . 31
4.2.10 Как переназначить стандартные потоки на другие устройства (например, файлы) без перекомпиляции? . . . 31
4.2.11 Что такое "кодирование символов"? Как Unicode решает проблему больших наборов символов? Структурное деление и максимальные codepoint'ы. . . . 32
4.2.12 Пример 8-битной кодировки, ограничения, и что определяет первую половину . . . 32
4.2.13 Пример кодировки с переменной длиной (UTF-8), преимущества и недостатки . . . 32
4.2.14 Типы для представления символов в C++ и литералы . . . 32
4.2.15 Пример статического C-массива и вычисление числа элементов (sizeof); decay массивов . . . 33
4.2.16 Связь между C-массивами и C-строками; можно ли по типу `char*` понять, C-массив это или C-строка? . . . 33
4.2.17 Что такое '\0' и отличие от '0'? . . . 33
4.2.18 Для чего используется `std::string`? Основные операции, инициализация, внутреннее представление . . . 33
4.2.19 Как прочитать токен и как прочитать строку с пробелами в `std::string`? 33
4.2.20 Связь между `sizeof(std::string)` и `s.length()`; что такое SSO . . . 34
4.2.21 Преобразования между строками и числами (в стиле C и C++) . . . 34
4.2.22 Как осуществляется конкатенация строк; что эффективнее . . . 34
4.2.23 Как `std::stringstream` помогает при множественной конкатенации и при преобразованиях . . . 34
4.2.24 Как `std::stringstream` используется в модульном тестировании? . . . 34
4.2.25 Какой статус `std::stringstream` в иерархии потоков STL? . . . 35

5 Функциональная декомпозиция, ссылки и константность 36
5.1 Темы . . . 36
5.2 Контрольные вопросы и ответы . . . 36
5.2.1 Как решить большую задачу так, чтобы решение было корректным, проверяемым, воспроизводимым, переиспользуемым, читаемым? . . . 36
5.2.2 Что такое функция с точки зрения языка C++? . . . 37
5.2.3 Как определить функцию / процедура? Какие основные компоненты функции? . . . 37
5.2.4 Что такое неполный тип `void`? Можно ли породить объект такого типа? А типа `void*`? . . . 37
5.2.5 Что такое свободная функция и что такое функция-член? Примеры. . . 37
5.2.6 Является ли `std::getline` свободной или членом? . . . 38
5.2.7 Является ли `static` функция, объявленная в классе, свободной? Как вызвать? . . . 38
5.2.8 Что такое side-effect у функции и почему это нежелательно для свободных функций? . . . 38
5.2.9 Что такое методы для классов и пример метода `std::string`? . . . 38
5.2.10 Что такое список формальных параметров функции? Можно ли составить список только из типов? . . . 38
5.2.11 Что такое сигнатура функции? Зачем её отделять от заголовка и прототипа? . . . 38

5.2.12 Чем различаются заголовок функции и прототип? Что относится к интерфейсной части? . . . 38
5.2.13 Может ли функция иметь более одной точки выхода? Хороший ли это стиль? Может ли `void` функция иметь `return`? . . . 39
5.2.14 Нужно ли компилятору полное определение функции? А линкеру? . . . 39
5.2.15 Как обычно распределяется интерфейс и реализация по заголовкам и модулем трансляции? . . . 39
5.2.16 Признаки хорошей функции (и объяснение) . . . 39
5.2.17 Симптомы плохой функции (smell code) . . . 39
5.2.18 Что такое реентерабельная функция? Пример параметризуемой и непараметризуемой функции . . . 40
5.2.19 Принцип высокой/низкой связности (coupling & cohesion) . . . 40
5.2.20 Тестируемость функции . . . 40
5.2.21 Оптимальный размер функции и ясность . . . 40
5.2.22 Принципы функциональной декомпозиции . . . 40
5.2.23 Принципы S.O.L.I.D. и примеры . . . 40
5.2.24 Чем отличаются фактические параметры (arguments) от формальных параметров? . . . 41
5.2.25 Что такое перегрузка функций и условия для перегрузок? . . . 41
5.2.26 Методы графического представления взаимодействия функций . . . 41
5.2.27 Что такое CV-квалификаторы? Что такое `const` и `volatile`? . . . 41
5.2.28 Что такое константный объект (const)? Есть ли у него имя, адрес, тип, значение? Что меняет `const`? . . . 42
5.2.29 Что такое ссылка на объект и зачем она нужна? Как задаётся? . . . 42
5.2.30 Почему говорят, что ссылка — «еще одно имя для объекта»? Примеры. 42
5.2.31 Можно ли перепривязать ссылку? Отложенное привязывание? . . . 42
5.2.32 Когда параметры передаются по ссылке? Чем отличаются ссылки от передачи по значению? . . . 42
5.2.33 Как вернуть из функции более одного значения (не используя массивы/структуры)? . . . 43
5.2.34 Что такое «Правило № 5»: «В C++ по умолчанию — всё по значению»? 43
5.2.35 Что такое константная ссылка и когда её использовать? . . . 43
5.2.36 Пример, где константная ссылка эффективна; когда предпочесть копирование . . . 43
5.2.37 Правило трёх указателей (ПТУ) — «большие» и «малые» объекты . . . 43
5.2.38 Примеры: когда передавать по значению / по неконстантной ссылке / по константной ссылке . . . 43
5.3 Короткий справочник-резюме (чтобы ничего не забыть) . . . 44

1 Вокруг языка C++

1.1 Темы

Материал этого раздела даёт обзор экосистемы вокруг языка C++ и включает следующие направления:

- Существующие стандарты и драфты языка, жизненный цикл стандартов.
- The **Zero Overhead Principle** — принцип нулевых накладных расходов и примеры его проявления в коде.
- Сборка и организация проектов: CMake — ключевые директивы и типичные сценарии.
- Ключевой принцип: *интерфейсы* и *реализации* (разделение обязанностей).
- Заголовочные файлы и модули трансляции: назначение, типичное содержимое и ограничения.
- Этапы построения программы (lifecycle): от исходников до исполняемого файла.
- Правила именования файлов (Правило № 0) и договорённости по стилю оформления кода.

1.2 Контрольные вопросы

1.2.1 Какие стандарты языка существуют? Каков цикл выпуска новых стандартов?

Кратко: язык C++ развивается стандартами, каждый стандарт — версия языка. Основные стандарты (хронология): C++98, C++03, C++11, C++14, C++17, C++20, C++23. Разрабатывает стандарт ISO WG21 (ISO/IEC JTC1/SC22/WG21), включая рабочие драфты и предложения.

Цикл выпуска: в последние годы крупные стандарты появляются примерно раз в 2–3 года (C++11, C++14, C++17, C++20, C++23). Между крупными версиями публикуются рабочие драфты и технические спецификации.

Примечание: между стандартами публикуются *Technical Specifications (TS)* и *Working Drafts* — экспериментальные/переходные документы, которые могут затем войти в стандарт.

1.2.2 Что такое The Zero Overhead Principle? Привести пример появления этого принципа в коде.

Определение: *Zero Overhead Principle* означает, что абстракции языка/библиотеки не должны навязывать прирост стоимости (время/память), которого нельзя избежать при реализации на более низком уровне. Иначе говоря, при разумном использовании абстракций вы должны получать производительность, сопоставимую с ручной реализацией.

Последствия:

- Абстракции должны быть проектированы так, чтобы компилятор мог устранять накладные расходы (инлайнинг, оптимизация шаблонов, `constexpr`).
- Если накладные расходы неизбежны, должна быть возможность выбрать более низкоуровневый механизм.

Пример:

```
template <typename T>
inline T add(T a, T b) { return a + b; }

int main() {
    int x = add(1, 2); // оптимизатор может полностью убрать вызов
}
```

Другой пример: итераторы STL для `std::vector<T>` при оптимизации компилируются в адресную арифметику (аналог работы с указателями) без дополнительных накладных расходов.

1.2.3 Какие основные директивы CMake/файлов проекта? Для чего нужна директива `add_executable`? Каковы её параметры?

Коротко о CMake: CMake — система генерации файлов сборки (build system generator). Часто используемые команды:

- `cmake_minimum_required(VERSION <x>)` — минимальная версия CMake.
- `project(<name> [LANGUAGES ...])` — объявление проекта.
- `add_executable(<name> [sources...])` — создание исполняемого таргета.
- `add_library(<name> [STATIC|SHARED|INTERFACE] [sources...])` — создание библиотеки.
- `target_include_directories(<target> ...)` — пути для заголовков.
- `target_link_libraries(<target> ...)` — связывание с библиотеками.
- `find_package(...)` — поиск внешних пакетов.
- `target_compile_features`, `target_compile_options` — установка флагов компиляции.
- `set(...)` — установка переменных.

add_executable: объявляет исполняемый таргет. Простейший синтаксис:

```
add_executable(<name> [WIN32] [MACOSX_BUNDLE] [EXCLUDE_FROM_ALL]
               source1 source2 ...)
```

Параметры:

- `<name>` — имя таргета.
- список исходников — имена файлов, которые войдут в таргет.
- опциональные флаги: `WIN32`, `MACOSX_BUNDLE`, `EXCLUDE_FROM_ALL`.

Пример:

```
cmake_minimum_required(VERSION 3.15)
project(myprog LANGUAGES CXX)

add_executable(myprog
    src/main.cpp
    src/foo.cpp
)

target_include_directories(myprog PRIVATE include)
target_compile_features(myprog PRIVATE cxx_std_17)
```

Замечание: в современном CMake рекомендуется работать через `target_*` интерфейсы и указывать видимость зависимостей: `PRIVATE`, `PUBLIC`, `INTERFACE`. Это управляет тем, какие флаги/директории видимы только для самого таргета и какие транзитивно передаются потребителям.

1.2.4 Что такое интерфейс и что такое реализация? Что из них идеальное, что материальное?

Интерфейс — формальное описание того, какие операции доступны внешнему коду: сигнатуры функций, объявленные типы, контракты поведения. Интерфейс отвечает на вопрос *что* делать.

Реализация — конкретный код, структуры данных и алгоритмы, которые исполняют интерфейс; отвечает на вопрос *как* делать.

Интерфейс чаще ассоциируется с идеальной спецификацией (идеальное), а реализация — с материальной частью (код, файлы, бинарники).

1.2.5 Примеры разделения на интерфейс и реализацию в C++

Классический header + source:

```
// include/foo.h  (интерфейс)
#pragma once
int add(int a, int b);

// src/foo.cpp (реализация)
#include "foo.h"
int add(int a, int b) { return a + b; }
```

Модули C++20 (пример):

```
// math.ixx  (модульный интерфейс)
export module math;
export int add(int a, int b);

// math_impl.cpp  (модульная реализация)
module math;
int add(int a, int b) { return a + b; }
```

В первых примерах интерфейс — заголовочный файл, в модулях — module interface unit (экспортируемая часть).

1.2.6 Для чего используются заголовочные файлы в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними нельзя? Какие расширения встречаются? Какова связь между заголовочными файлами и принципом интерфейс–реализация?

Назначение: заголовочные файлы (headers) служат для объявления интерфейса: декларации функций, классов, шаблонов, типов и констант.

Обычно помещают:

- объявления типов и классов (обычно лишь публичную часть),
- прототипы функций,
- шаблоны (полностью),
- inline/constexpr функции (которые должны быть видимы при инстанциации),
- using/typedef, enum, макросы (по необходимости).

Чего обычно не кладут:

- большие реализации (если не inline/шаблон),
- определения глобальных переменных (вместо этого используют `extern` в заголовке и единственное определение в `.cpp`).

Расширения: `.h`, `.hpp`, `.hh`, `.hxx` — проектная конвенция.

Связь с интерфейсом/реализацией: заголовочный файл обычно содержит интерфейсную часть; реализация хранится в .cpp (или в модуле реализации). Это реализует принцип разделения обязанностей и инкапсуляцию.

1.2.7 Для чего используются модули трансляции в C++? Что в них обычно помещают? Что с ними делают и чего нельзя? Какие расширения встречаются? Связь с интерфейс-реализацией?

Назначение модулей:

- устранение недостатков препроцессорной модели `#include` (меньше дублированных зависимостей, быстрее компиляция),
- явное разделение интерфейса (export) и реализации,
- более строгая и быстрая модель управления зависимостями.

Обычно помещают:

- экспортируемые объявления в module interface unit,
- приватные реализации в модульных реализациях или в той же единице, но без `export`.

Чего избегают / ограничения:

- не все макросы и/или препроцессорные приёмы совместимы с модульной организацией (модули и препроцессор взаимодействуют иначе),
- практика и детали зависят от компилятора; сборка модулей добавляет артефакты (BMI — binary module interface) и порядок сборки становится важен.

Расширения (конвенции): `.ixx`, `.cppm`, `.mpp` и т.п. — соглашения сообщества/компиляторов.

Связь с интерфейсом/реализацией: модули предоставляют современный механизм явного разделения интерфейса и реализации, часто с лучшими гарантиями по времени сборки и изоляции символов.

Дополнительно: Этапы построения программы (кратко)
Этапы:

1. **Препроцессинг** (`#include`, `#define`, условная компиляция) — текстовые преобразования (для классических includes).
2. **Компиляция** — перевод единицы трансляции в объектный код (парсинг, семантика, оптимизации).
3. **Ассемблирование** — если необходима отдельная стадия.
4. **Линковка** — связывание объектных модулей и библиотек, формирование исполняемого файла или библиотеки.

Правила именования файлов (Правило #0) и договорённости по стилю оформления кода

- Принцип: согласованность важнее выбора конкретного правила.
- Общие рекомендации:
  - исходники: `.cpp`, заголовки: `.h` / `.hpp`;
  - имена файлов маленькими буквами, разделение слов подчёркиванием или дефисом по соглашению проекта;
  - структура каталогов должна отражать неймспейсы/модули;
  - header-guard'ы формируются с учётом полного пути и имени проекта, например `PROJECT_DIR_SUBDIR_FILENAME_H`.
- Стилевые договорённости:
  - единый стиль отступов/имен (camelCase или snake_case);
  - интерфейс отделяется от реализации;
  - минимизация области видимости (anonymous namespaces, internal linkage);
  - документация контрактов интерфейса (пре/постусловия).

1.3 Примеры и заметки

1.3.1 Header guard (классическая реализация)

```
#ifndef PROJECT_SUBDIR_FOO_H
#define PROJECT_SUBDIR_FOO_H

int add(int a, int b);

#endif // PROJECT_SUBDIR_FOO_H
```

1.3.2 Пример с `pragma once`

```
#pragma once
```

1.3.3 Короткие практические рекомендации

- Уметь формулировать разницу интерфейс vs реализация и приводить конкретные примеры (header/`.cpp`, module interface/implementation).
- Привести как минимум один пример, показывающий Zero Overhead Principle (шаблоны + inline + `constexpr`).
- Знать ключевые команды CMake и смысл таргет-ориентированного подхода (`target_include_di` `target_link_libraries`, квалификаторы `PRIVATE/PUBLIC/INTERFACE`).
- Понимать этапы построения программы и роль препроцессора; уметь объяснить отличия модулей от механики `#include`.

2 Основы C++

2.1 Темы

Материал этого раздела преимущественно рассматривается в рамках лекций 2–3. Включает:

- Препроцессор языка C(++): назначения, этапы работы в жизненном цикле построения программы, директивы.
- Применение препроцессора для конфигурирования текста программы для конфигураций `debug` и `release`. Пример с `cassert`.
- «Листинг» программы как вывод препроцессора, содержание.
- Директивы условного включения: `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif`, `#endif`.
- Директива включения заголовочного файла `#include`. Системные (библиотечные) и пользовательские (проектные) заголовки, разница в синтаксисе. Порядок включения заголовков (стилевая договорённость).
- Проблема повторного (косвенного) включения заголовочного файла.
- Header guard и его классическая реализация. Правила именования header guard с привязкой к пути к файлу и имени проекта. Реализация с помощью `#pragma once`.
- Заголовочный файл модуля как интерфейсная часть модуля.
- Принцип описания (declaration) и определения (definition). Связь описания и определения с интерфейсом и реализацией.

2.2 Контрольные вопросы и развёрнутые ответы

2.2.1 Каково назначение препроцессора языка C(++)?

Препроцессор выполняет текстовые преобразования исходного кода до этапа компиляции. Его назначение:

- Подстановка содержимого файлы при директиве `#include`.
- Управление условной компиляцией (`#if`/`#ifdef`/`#else`...).
- Подстановка макросов (`#define`).
- Генерация конфигурационных вариантов исходного текста (напр., `debug` vs `release`).

Важно: препроцессор оперирует текстом, а не семантикой языка C++; для него синтаксис и семантика C++ не имеют принципиального значения — он выполняет простые текстовые трансформации.

2.2.2 С какими сущностями манипулирует препроцессор? Имеет ли для него принципиальное значение синтаксис и семантика языка C++?

Препроцессор манипулирует:

- макросами: `#define NAME replacement`,
- условной компиляцией: `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, `#endif`,
- включениями файлов: `#include`,
- директивами типа `#error`, `#pragma`.

Препроцессор не проверяет семантику C++ — он лишь подготавливает текст, который потом парсит компилятор. Поэтому конструкции препроцессора могут нарушить синтаксис C++ если их неправильно использовать (например, незакрытые `#if`).

2.2.3 На примере директивы `assert` покажите, как препроцессор меняет код программы для конфигураций сборки `debug` и `release`.

Стандартная макрос-функция `assert` из `<cassert>` определяется примерно так: если `NDEBUG` не определён, `assert(expr)` проверяет выражение, иначе — превращается в пустой макрос.

Пример кода:

```
#include <cassert>

void f(int x) {
    assert(x != 0);
    // ...
}
```

При сборке в `debug` (по умолчанию `NDEBUG` не определён), макрос `assert` оставляет проверку, и в случае `x == 0` программа будет аварийно завершена с диагностикой. При сборке в `release` часто определяют `NDEBUG` (например, через флаг компилятора `-DNDEBUG`), и тогда макрос `assert` разворачивается в пустой код — проверка полностью исключается препроцессором и не влияет на бинарник.

Иллюстрация (что делает препроцессор): команда

```
g++ -E file.cpp -o file.i
```

создаст префиксированный листинг `file.i` с результатом подстановок, где видна либо проверка, либо её отсутствие в зависимости от `NDEBUG`.

2.2.4 Что содержит и что не содержит листинг (результат работы препроцессора)?

Листинг (output of preprocessor) содержит:

- исходный код с подставленными содержимыми включённых файлов,
- развёрнутые макросы (текст замены),
- удалённые комментарии (обычно),
- результаты условной компиляции (фрагменты, оставшиеся после `#if/#endif`).

Листинг не содержит:

- семантической информации компилятора (типов, связываний) — это задача компилятора;
- исходных комментариев (они обычно удаляются препроцессором);
- макросов как таковых — они развёрнуты (заменены).

2.2.5 Приведите пример конфигурирования текста исходного кода программы условными директивами препроцессора (напр., включения фрагмента кода, релевантного определённой ОС).

Пример условного включения кода для Windows vs POSIX:

```
#ifdef _WIN32
    // код, специфичный для Windows
    #include <windows.h>
#elif defined(__linux__)
    // код, специфичный для Linux
    #include <unistd.h>
#else
    // код для прочих платформ
#endif
```

Такой подход позволяет поддерживать кроссплатформенный исходник, выбирая нужные куски кода в зависимости от предопределённых макросов компилятора.

2.2.6 Как включить в модуль системный заголовочный файл? — и заголовочный файл из проекта?

- Системный (библиотечный) заголовок: `#include <vector>`. Компилятор ищет такие файлы в системных/установленных каталогах.
- Проектный заголовок: `#include "foo.h"`. Сначала ищется относительно каталога исходного файла, затем в системных каталогах (поведение зависит от компилятора/-флагов).

Стилевая договорённость: сначала включать стандартные/системные заголовки, затем сторонние (third-party), затем проектные заголовки — чтобы избежать скрытых зависимостей и гарантировать, что заголовки проекта сами корректно включают необходимые зависимости.

2.2.7 В чем заключается проблема повторного (косвенного) включения заголовочного файла? Приведите пример.

Проблема: если один и тот же заголовочный файл включается несколько раз в пределах одной единицы трансляции (прямо или косвенно через другие заголовки), то без защиты это приведёт к множественным определениям (например, повторному объявлению классов, перегрузкам define и т.п.), что вызывает ошибки компиляции (множественное определение или синтаксические ошибки).

Пример:

```
// a.h
struct A { int x; };

// b.h
#include "a.h"

// c.cpp
#include "a.h"
#include "b.h" // a.h будет включён повторно без защиты -> ошибки
```

2.2.8 Что такое header guard и какую проблему он призван предотвратить? Приведите конкретный пример.

Header guard — условная компиляция вокруг содержимого заголовочного файла, предотвращающая его повторное включение.

Классическая реализация:

```
// foo.h
#ifndef PROJECT_SUBDIR_FOO_H
#define PROJECT_SUBDIR_FOO_H

struct Foo { int x; };

#endif // PROJECT_SUBDIR_FOO_H
```

Если `foo.h` включён несколько раз, первый проход определяет макрос `PROJECT_SUBDIR_FOO_H`, а последующие проходы содержимое между `#ifndef` и `#endif` пропускают — повторное объявление не происходит.

2.2.9 Как правильно именовать header guard (стилевые договорённости)?

Рекомендации:

- Включать в имя guard путь к файлу и имя проекта, переводя символы пути в подчёркивания и приводя к верхнему регистру, например: `MYPROJ_MODULE_SUBMODULE_FOO_H`.
- Это снижает риск коллизий между одинаковыми именами файлов в разных частях проекта или в подключённых библиотеках.

2.2.10 Реализация с помощью `#pragma once`

Альтернатива header guards:

```
// foo.h
#pragma once

struct Foo { int x; };
```

`#pragma once` проще и короче; поддерживается всеми современными компиляторами, но формально это не часть стандарта C++ (хотя де-факто повсеместно доступно). Некоторые проекты используют и то и другое для максимальной совместимости.

2.2.11 Можно ли использовать заголовочный файл без header guard и если да, то какие ограничения это накладывает?

Технически — можно, но тогда:

- файл не должен содержать определений, которые при множественном включении приведут к ошибкам (например, глобальные переменные с выделением памяти, определения классов без `inline` и т.д.);
- обычно это означает, что такой заголовок может содержать только *inline* функции, шаблоны (которые допускают множественное включение), константы с внутренней связностью (например, `static const`) и т.п.

Практически всегда рекомендуется использовать header guards или `#pragma once`.

2.2.12 Что обычно кладут в заголовочный файл — интерфейсную часть некоторого модуля `foo`? А что обычно не кладут — и почему?

Обычно кладут в `foo.h`:

- объявления классов/структур и их публичных интерфейсов,
- прототипы (declarations) функций,
- объявления глобальных констант/типов (через `extern` при необходимости),
- inline-функции и шаблоны (которые требуют видимости при инстанциации).

Обычно не кладут:

- большие реализации (тяжёлые определения функций, если они не `inline`), чтобы не увеличивать время компиляции и избежать множественных определений;
- определения глобальных переменных (за исключением `inline` variables в C++17) — вместо этого используют `extern` в заголовке и определение в `.cpp`.

2.2.13 Чем в C++ описание (declaration) принципиально отличается от определения (definition)? Как можно наблюдать эффект описания и эффект определения?

Описание (declaration) сообщает компилятору о существовании сущности: её имени и типа (например, прототип функции, объявление `extern int x;`). Описание позволяет компилятору проверить корректность вызовов и ссылок на сущность без знания её реализации.

Определение (definition) создаёт саму сущность: выделяет память для объекта или предоставляет тело функции (например, `int x = 0;` или `int f()  return 42;`).

Наблюдение эффектов:

- Если есть только описание (прототип) и отсутствует определение при линковке вызова — линковщик выдаст ошибку undefined reference.
- Если есть определение, но код ссылается на него правильно — всё линкуется успешно.
- Компилятор требует описания для корректной генерации кода вызова; линкер требует определения для разрешения символов.

2.3 Короткие примеры (безопасные для вставки в .tex)

Пример использования условной компиляции для assert

```
#include <cassert>

void f(int x) {
    assert(x != 0); // в релизе при -DNDEBUG эта проверка исчезает
}
```

Пример header guard и pragma once

```
// include/project/sub/foo.h
#ifndef PROJECT_SUB_FOO_H
#define PROJECT_SUB_FOO_H

struct Foo { int x; };

#endif // PROJECT_SUB_FOO_H

// или кратко:
#pragma once
```

Пример порядка include (рекомендация)

```
// recommended order in .cpp:
#include <iostream>   // стандартные заголовки
#include <vector>     // ...
#include "thirdparty/foo.h" // сторонние библиотеки
#include "project/sub/foo.h" // собственные заголовки
```

2.4 Итог и рекомендации для экзамена / коллоквиума

- Уметь объяснить, что делает препроцессор и привести пример условной компиляции (например, `NDEBUG` + `assert`).
- Понимать, что листинг препроцессора (`g++ -E`) показывает развёрнутый исходный текст.
- Уметь показать классический header guard и объяснить, почему имя guard придумывают с привязкой к пути и проекту.
- Знать разницу между `#include <...>` и `#include "..."` и рекомендованный порядок include.
- Чётко различать declaration и definition — где они появляются (заголовок vs `.cpp`) и чего требует компилятор, а чего — линкер.

3 Базовые конструкции C++

3.1 Темы

Материал этого раздела преимущественно рассматривается в рамках лекций 2–4. Включает следующие направления:

- Алфавит языка C++.
- Простая программа на C++: структура, основные действующие лица.
- Ключевые слова; важнейшие ключевые слова (включая новшества C++11+).
- Идентификаторы, правила их составления; влияние кодстайла.
- Инструкция (statement) — понятие, классификация.
- Инструкции описания (declaration): объектов данных, функций; ключевое слово `extern`; прототип функции.
- Инструкции определения (definition) и описание с инициализацией.
- Варианты инициализации объектов: `=`, `()`, списковая инициализация {}, различия, narrowing.
- Блок инструкций, область видимости (scope).
- Функция как именованный блок инструкций: интерфейс и реализация, сигнатура.
- Объекты и типы: характеристики, представление в памяти, `sizeof`.
- Классификации типов: классическая трёхтаксонная и двухтаксонная (cppreference).
- Числовые типы: целые (знаковость, размеры, `<cstdint>`, `<climits>`), плавующие.
- POD-типы и их особенности.
- Выражения и операторы: арность, приоритет, ассоциативность; инструкция-выражение.
- Управляющие конструкции: ветвления (`if`, `switch`), тернарный оператор, циклы (`while`, `for`, range-based `for`, `do..while`), `break`/`continue`/`return`.

3.2 Контрольные вопросы и развернутые ответы

3.2.1 Какие ключевые слова введены/важны (особенно с C++11 и новее) и каково их назначение?

Ниже перечислены ключевые слова, которые особенно важны начиная с C++11, с краткой ролью:

- `auto` — автоматическое выведение типа для переменных и возвращаемых значений функций (синтаксически упрощает код, особенно с шаблонными типами).
- `nullptr` — литерал нулевого указателя, безопасная замена `NULL` / `0`.
- `constexpr` — константные выражения, вычисляемые на этапе компиляции при возможности; позволяет писать вычисления в контексте компиляции.

- `decltype` — оператор для получения типа выражения.
- `override`, `final` — спецификаторы виртуальных методов (проверка корректности переопределения, запрет дальнейшего переопределения).
- `thread_local` — спецификатор для переменных с локальным потоком хранения.
- `noexcept` — спецификация, указывающая, что функция не выбрасывает исключений (может влиять на оптимизации и семантику move).
- `static_assert` — статическая проверка условий на этапе компиляции.
- `alignas`, `alignof` — управление выравниванием объектов.

3.2.2 Какова базовая структура программы на C++? Какие основные компоненты и на каком уровне видимости встречаются? Какие из них обрабатываются какими инструментами построения программы?

Базовая структура:

- **Препроцессорные директивы** (`#include`, `#define`) — текстовая вставка/макросы, обрабатываются препроцессором.
- **Декларации и определения** (функций, классов, глобальных объектов) — обрабатываются компилятором в единицах трансляции.
- **main** — точка входа программы (функция `int main(...)`).
- **Пространства имён** (`namespace`) — логическая группировка имён.

Инструменты:

- Препроцессор обрабатывает `#include` и макросы.
- Компилятор операции синтаксического и семантического анализа, генерации объектного кода для каждой единицы трансляции.
- Линкер связывает объектные модули и библиотеки.

Уровни видимости:

- **scope блока** (локальные переменные).
- **function scope** (имена параметров, локальные метки).
- **namespace scope** (глобальные имена видимые в неймспейсе).
- **class scope** (члены класса).

3.2.3 Какие идентификаторы корректны для объектов, типов, функций, глобальных констант, символов препроцессора? Как влияют кодстайл договорённости?

Правила (общие):

- Идентификатор начинается с буквы (A–Z, a–z, _ (нижнее подчёркивание)); далее буквы, цифры или _.
- Не может совпадать с ключевым словом.
- Имена, начинающиеся с одного подчёркивания и большого символа или с двух подчёркиваний, зарезервированы для реализации (стандарт).

Рекомендации по кодстайлу (проектные договорённости):

- `snake_case` для переменных/функций или `camelCase` — выбирать одно.
- Типы и классы часто пишут в `PascalCase` или с префиксом (внутренние соглашения).
- Глобальные константы часто в `UPPER_SNAKE_CASE` или в `constexpr` внутри namespace.
- Символы препроцессора (макросы) обычно в `UPPER_SNAKE_CASE`.

3.2.4 Что такое инструкция (statement)? Какой признак указывает на появление инструкции? Примеры исключений.

Инструкция — завершённое предложение языка, выполняемое во время исполнения; обычно заканчивается точкой с запятой `;`. Примеры: выражение- инструкция (`x++;`), инструкция присваивания, инструкция возврата (`return`), инструкция выбора (`if`), цикл (`for`), блок ({ ... }).

Исключения:

- Объявления (декларации) иногда не заканчиваются `;` (например `if (int x = f()) ...` — объявление внутри заголовка `if`).
- Блоки сами по себе — инструкция без завершающей точки с запятой.

3.2.5 Что такое инструкция описания объекта? Пример.

Описание (declaration) сообщает компилятору о существовании сущности без её полной реализации. Пример:

```
extern int global_value; // описание: говорит о существовании с внешней связью
```

3.2.6 Что такое инструкция описания функции? Пример.

Прототип (declaration):

```
int add(int a, int b); // прототип функции - описание интерфейса
```

3.2.7 Что такое инструкция определения и описания объекта? Пример.

Определение выделяет хранилище/реализует сущность:

```
int global_value = 42; // определение и инициализация
```

3.2.8 Может ли существовать только описание без определения и наоборот? Примеры.

Да.

- Только описание: `extern int x;` — требуется определение в другом модуле.
- Только определение без предварительного описания: `int x = 0;` — это одновременно и определение, и описание.

Для функции:

```
int f(); // описание (прототип)
... (где-то else)
int f() { return 1; } // определение
```

3.2.9 Как производится инициализация объектов и когда? Варианты и различия.

Инициализация происходит в момент определения объекта (или при создании временного объекта). Основные формы:

- copy-initialization: `T x = a;`
- direct-initialization: `T x(a);` или `T x{a};`
- list-initialization (унифицированная): `T x{a, b};` или `T x = {a,b};`
- aggregate initialization: `S s{1,2};` для aggregate.

Общее: инициализация задаёт начальное значение/состояние объекта. Различие:

- `{}` предотвращает narrowing (в большинстве случаев) и предпочтительна для безопасной инициализации.
- `=` позволяет неявные преобразования, включая narrowing при приведениях.

3.2.10 Что такое унифицированный (list-)инициализатор? Когда рекомендуется применять?

Унифицированный инициализатор — это синтаксис `{ ... }`. Рекомендуется применять для:

- предотвращения нежелательного сужения (narrowing),
- явной инициализации aggregate-типов,
- единообразного стиля.

3.2.11 Пример инструкции определения и описания с инициализацией. Когда это важно?

```
static const int MAX = 100; // определение и инициализация (видимо глобальная константа)
struct Point { int x, y; };
Point p{1, 2}; // aggregate initialization
```

Важно: для глобальных констант в заголовках использовать `inline constexpr` или `extern` + определение в .cpp, чтобы избежать множественного определения.

3.2.12 Блок инструкций: зачем и можно ли использовать блок без заголовка?

Блок создаёт локальную область видимости и ограничивает время жизни переменных:

```
{
    int tmp = compute();
    // tmp недоступен за пределами блока
}
```

Да, блок без заголовка можно использовать для локализации временных переменных или ограничивания области видимости ресурсов (RAII).

3.2.13 Как блок влияет на область видимости?

Переменные, объявленные в блоке, видимы только внутри него; они уничтожаются при выходе из блока. Это позволяет контролировать время жизни и избегать конфликтов имён.

3.2.14 Как в C++ определить именованный блок инструкций (функцию)?

Функция — именованный блок с параметрами:

```
int sum(int a, int b) { return a + b; }
```

Параметры задают входные данные; возвращаемое значение — выход.

3.2.15 Объекты и типы: связь, характеристики объектов и типов.

- **Объект** — region of storage with an associated type and value.
- **Тип** определяет представление и допустимые операции (семантика).
- Характеристики объекта: адрес (если применимо), время жизни, выравнивание, размер, значение.
- Характеристики типа: whether it is POD, trivial, standard-layout, signed/unsigned, arithmetic/aggregate, etc.

3.2.16 Может ли существовать объект без типа?

Нет в традиционном C++: каждый объект имеет тип. Исключения — низкоуровневые конструкции (например, память, выделенная через `void*`), но сам `void*` не является объектом, это указатель на неуказанный тип.

3.2.17 Как объекты представляются в оперативной памяти? Какие характеристики влияют?

Представление зависит от:

- типа (фундаментальный/производный/пользовательский),
- размера (`sizeof`),
- выравнивания (`alignof`),
- кодировки для целых/символьных типов и представления для float/double,
- конвенций платформы (endianness — обычно little-endian на x86).

3.2.18 Оператор `sizeof`: применение и возвращаемый тип

- `sizeof(T)` или `sizeof expr` возвращает значение типа `std::size_t`.
- Часто используется для определения количества элементов в статическом массиве: `sizeof arr / sizeof arr[0]`.
- Результат обычно вычисляется на этапе компиляции (если тип известен и не VLA).

3.2.19 Что такое операционная семантика типа? ADT примеры

Операционная семантика — набор операций, определённых для типа, и их поведение (например, для `int`: +, -, *, /, побитовые операции; для `std::string`: конкатенация, длина, доступ по индексу). ADT (Abstract Data Type) — стек, очередь, словарь, множество и т.д.; примеры: `std::vector<T>` как ADT динамического массива.

3.2.20 Классификация типов в C++ (трёхтаксонная и двухтаксонная)

- Трёхтаксонная (классическая): фундаментальные (int, char, float...), производные (указатели, массивы, ссылки, функции), пользовательские (class/struct/enum).
- Двухтаксонная (cppreference): фундаментальные и производные (включает пользовательские в производные/фундаментальные по контексту).

3.2.21 Модификаторы типов для производных типов

Примеры: указатель (`T*`), ссылка (`T&`), массив (`T[N]`), функция типа (`T(Args...)`), cv-квалификаторы (`const`, `volatile`).

3.2.22 Целочисленные типы: какие и как определяется знаковость, диапазоны

- Базовые типы: `char`, `short`, `int`, `long`, `long long` (со знаковостью implementation-defined для `char`).
- Для фиксированных размеров: `int8_t`, `int16_t`, `int32_t`, `int64_t` в `<cstdint>` (если доступны).
- Диапазоны можно узнать через `<climits>` (макросы) или `std::numeric_limits<T>::min()/max(`
- Знаковость: `signed` или `unsigned`; операции и преобразования зависят от этого.

3.2.23 Типы с плавающей точкой: структура и специальные значения

Типы: `float`, `double`, `long double`. Специальные значения: `NaN`, `+inf`, `-inf`, представляются согласно IEEE-754 на большинстве платформ.

3.2.24 Инициализация числовых объектов, литералы, narrowing

- Литералы: `42` (int), `42u` (unsigned), `42L` (long), `3.14` (double), `3.14f` (float).
- Narrowing: потеря точности/диапазона при преобразовании (например, `double -> int` или `long -> int`).
- Списковая инициализация `{}` запрещает narrowing в большинстве ситуаций (даёт ошибку компиляции).

3.2.25 Что такое POD-тип? Примеры и особенности

POD (Plain Old Data) — историческое понятие; в C++11 POD означает тип, который одновременно *trivial* и *standard-layout*. Примеры: простые структуры без пользовательских конструкторов, виртуальных функций или нестандартного размещения полей:

```
struct S { int a; double b; }; // POD в большинстве случаев
```

Особенность: POD-типы совместимы с C-структурами, их можно бинарно копировать без вызова конструкторов.

3.2.26 Что такое выражение, оператор, операнд? Может ли выражение состоять из одного операнда/оператора?

- **Выражение** — комбинация операндов и операторов, дающая значение/эффект.
- **Операнд** — объект, литерал, функция, результат другого выражения.
- **Оператор** — синтаксическая конструкция, выполняющая операцию (например, `+`, `-`, `*`, `?:`).
- Выражение может состоять из одного операнда (например, `x`), или одного оператора в случае унарных операторов (`-x`).

3.2.27 Классификация операторов по арности и приоритетности

- Арность: унарные, бинарные, тернарный (только `?:`).
- Приоритет (precedence) и ассоциативность определяют порядок вычисления; при необходимости используют скобки `(...)` для явного порядка.

3.2.28 Охарактеризуйте оператор (на примере): возвращаемое значение и случаи отсутствия возвращаемого значения

Разные операторы имеют разную семантику:

- `sizeof` — возвращает `std::size_t` (имеет значение при компиляции).
- Присваивание (`=`) — возвращает ссылку на левый операнд (lvalue), поэтому можно цепочить: `a = b = 0;` .
- Операторы управления потоком (`if`, `for`) не являются выражениями и не возвращают значение.

3.2.29 Постройте дерево разбора для выражения (пример)

Для выражения: `a + b * c - d` дерево разбора (с учётом приоритета `*` > `+` = `-`):

```
      (-)
     /   \
   (+)    d
  /   \
 a    (*)
      / \
     b   c
```

Это показывает, что сначала вычисляется `b * c`, затем `a + (b*c)`, затем результат минус `d`.

3.2.30 Операция присваивания. Различие между инициализацией и присваиванием

- **Инициализация** задаёт начальное значение при создании объекта: `int x = 5;` или `int x{5};`
- **Присваивание** изменяет значение уже существующего объекта: `x = 6;`
- Разница важна для объектов с конструкторами, для const-объектов (нельзя присваивать после инициализации) и для ресурсов (RAII).

3.2.31 Что такое инструкция-выражение?

Инструкция-выражение (expression-statement) — выражение, за которым следует `;`:

```
i += 1; // инструкция-выражение
```

3.2.32 Как работает `if` (короткий, длинный, с инициализацией)?

- **Короткий:** `if (cond) stmt;`
- **Длинный:** `if (cond) stmt1 else stmt2;`
- **C++17: if с инициализацией:** `if (Type var = expr; cond(var)) stmt;`

В C++17 можно объявлять переменную прямо в header `if`, видимую только в условии и теле.

3.2.33 Как работает `switch`? Зачем `break` и когда его не использовать?

`switch` сравнивает целочисленное или перечислимое выражение с `case` константами. `break` предотвращает *fall-through* (проваливание) к следующему case. Иногда fall-through желателен (например, группировка случаев):

```
switch (x) {
  case 1:
  case 2:
    handle_1_or_2();
    break;
  case 3:
    handle_3();
    // fall-through intentionally not used here if we want separate behavior
}
```

3.2.34 Соотношение тернарного оператора `?:` и `if`

- `?:` — выражение, возвращает значение, удобно в контекстах, где ожидается выражение.
- `if` — инструкция, позволяет выполнять сложные блоки, несколько инструкций и т.д.
- Ограничение `?:`: оба ветвящихся операнда должны быть совместимых типов; нельзя легко выполнять несколько инструкций без объединения в блок/функцию.

3.2.35 Цикл `while`: назначение и особенности

`while (cond) stmt;` — выполняет `stmt` пока `cond` истинно. Удобно для ситуаций, где количество итераций заранее неизвестно.

3.2.36 Цикл `for` и range-based `for`

- Классический `for`: `for (init; cond; incr) body;`
- Range-based (C++11): `for (auto& elem : container) body;` — удобен для перебора коллекций.

Эквивалент range-based `for` классическим:

```
std::vector<int> v = {1,2,3,4};
// range-based
for (auto &x : v) { use(x); }

// классический (итераторы)
for (auto it = v.begin(); it != v.end(); ++it) {
    auto &x = *it;
    use(x);
}

// или индексный, если допустимо:
for (std::size_t i = 0; i < v.size(); ++i) {
    auto &x = v[i];
    use(x);
}
```

Требование: контейнер должен быть *идентифицируемым* через `begin()/end()` (ADL допускает свободные begin/end), либо быть массивом с допустимым порядком доступа.

3.2.37 Цикл `do..while`: особенности

`do body while (cond);` — гарантированно выполняет тело хотя бы один раз. Удобен, если условие должно проверяться после первой итерации.

3.2.38 Может ли описание существовать без определения и наоборот?

Уже обсуждалось: описание (declaration) может существовать отдельно (прототипы, `extern`); определение должно присутствовать при линковке (либо в текущем модуле, либо в подключаемых библиотеках).

3.2.39 Как с помощью блока преодолеть ограничение одной инструкции, например в теле `if`?

В теле `if` требуется одна инструкция; чтобы выполнить несколько, используют блок:

```
if (cond) {
    stmt1;
    stmt2;
}
```

3.2.40 Разница между `short` и `int` кроме размера?

Принципиально различие — не строгое, но важные отличия:

- Семантика арифметики — промоции целых: при арифметике типы `short`, `char` обычно продвигаются до `int` (integral promotion), что влияет на вычисления и перегрузку функций.
- Диапазон и поведение при переполнении (для знаковых типов переполнение неопределено).

Пример: если `short a = 30000; short b = 30000; int sum = a + b;` промежуточное поведение в `int` важно для предотвращения переполнения.

3.2.41 Разница между `char` и `int` кроме размера?

Да:

- `char` может быть `signed` или `unsigned` в зависимости от реализации; это влияет на сравнения и расширение при промоции.
- `char` часто используется для хранения символов и кодировок; `int` — для арифметики.
- При выражениях `char` обычно продвигается до `int` при выполнении арифметики.

Пример: `char c = '\xFF'; if (c < 0) ...` — поведение зависит от того, является ли `char` знаковым.

Краткие практические советы для подготовки:

- Уметь различать описание и определение; знать роль `extern` и прототипов.
- На практике демонстрировать разные виды инициализации и объяснять, почему `{}` предпочтительнее для предотвращения narrowing.
- Понимать поведение `sizeof` и операции промоции целых.
- Быть готовым показать эквивалент range-based `for` через итераторы/индексы.
- Уметь объяснить, какие выражения дают значение и какие нет (операторы vs инструкции).

4 Потоки и строки STL

4.1 Темы

Материал этого раздела преимущественно рассматривается в рамках лекций 4–5. Включает:

- Вводовыводная модель консольного приложения (stdin/stdout).
- Понятие потока в STL: `std::istream`, `std::ostream`, потомки потоков.
- Операторы ввода/вывода » и « и их поведение с пробелами.
- Состояние потока: флаги `good`, `eof`, `fail`, `bad`; методы `good()`, `eof()`, `fail()`, `bad()`, `clear()`, `setstate()`.
- Идиомы последовательной вычитки (цикл while с `getline` и с оператором »).
- Перенаправление стандартных потоков на другие устройства (файлы) во время исполнения.
- Строки и кодировки: ASCII, Unicode, UTF-8; типы `char`, `char16_t`, `char32_t`, `wchar_t`, литералы `u8, u, U, L`.
- С-массивы, С-строки (нулевой терминатор '\0'), decay массива в указатель, вычисление размера через `sizeof`.
- Класс `std::string`: инициализация, методы, гарантия `.c_str()` на непрерывность и завершающий нулевой символ (с C++11), short-string optimization.
- Строковый поток `std::stringstream`: сборка строк, парсинг чисел, тестирование без файлов.

4.2 Контрольные вопросы и ответы

4.2.1 Что подразумевают под моделью ввода-вывода консольного приложения (CLI-application)? Какие основные компоненты ввода-вывода задействуются для него?

Модель CLI-приложения подразумевает, что программа взаимодействует с пользователем или окружением через текстовые/байтовые потоки: стандартный ввод (stdin), стандартный вывод (stdout) и стандартный поток ошибок (stderr). Основные компоненты:

- Устройства ввода/вывода (терминал, перенаправлённые файлы, пайпы).
- Библиотечные абстракции (в C++: `std::istream`, `std::ostream` и их конкретные потомки).
- Форматирование/разбор данных (операторы «/», вспомогательные функции).

4.2.2 Что такое стандартный поток ввода и стандартный поток вывода? Какими объектами они представлены? Какая область видимости этих объектов?

Стандартные потоки:

- `std::cin` — стандартный поток ввода, объект типа `std::istream`.
- `std::cout` — стандартный поток вывода, объект типа `std::ostream`.
- `std::cerr` / `std::clog` — потоки ошибок/логирования (объекты `std::ostream`).

Эти объекты объявлены в заголовке `<iostream>` и находятся в пространстве имён `std`. Они описываются (declaration) в заголовках библиотеки и определяются (definition) в реализациях стандартной библиотеки (бинарных/линкованных объектах).

4.2.3 Какой заголовочный файл необходим для доступа к стандартным потокам? Где описаны и где определены эти объекты?

Подключать нужно:

```
#include <iostream>
```

Описание (прототипы/extern) потоков даётся в заголовке; фактические определения находятся в реализации стандартной библиотеки, поставляемой с компилятором/системой (они линкуются при сборке).

4.2.4 Какие типы данных у стандартных потоков и какая иерархия наследования?

Типы потоков основаны на шаблонах:

- `std::basic_ios<CharT,Traits>` — базовая инфраструктура.
- `std::basic_istream<CharT,Traits>` и `std::basic_ostream<CharT,Traits>` — ввод/-вывод.
- `std::basic_iostream<CharT,Traits>` — наследует и ввод, и вывод.
- Специальизации для char: `std::istream`, `std::ostream`, `std::iostream`.
- Потомки: `std::ifstream`, `std::ofstream` (файловые); `std::istringstream`, `std::ostringstrea` `std::stringstream` (строковые).

Ромбовидное наследование (Deadly Diamond): `basic_iostream` наследует от `basic_istream` и `basic_ostream`, которые оба используют общую базу `basic_ios`. В стандартной библиотеке это организовано корректно, чтобы не создавать конфликтов; концептуальная проблема «алмазного» наследования показывает, почему нужно аккуратно проектировать и использовать виртуальные базовые классы или хорошо продуманную иерархию.

4.2.5 Какие операции ввода/вывода используются (операторы » и «)? Можно ли их перепутать? Как они работают с пробелами? Что возвращают эти операторы?

- Оператор `operator»` читает следующий токен (по умолчанию разделитель — пробельные символы) и преобразует в запрошенный тип (при возможности).
- Оператор `operator«` записывает в поток представление объекта (поток форматируется с учётом флагов).
- Нельзя использовать оператор » для объектов `std::ostream` или « для `std::istream`; они перегружены для правильных комбинаций типов.
- При чтении строк с пробелами » остановится на первом пробельном символе; для чтения строки с пробелами используют `std::getline`.
- Возвращаемое значение у » и « — ссылка на поток (`std::istream&` или `std::ostream&`), что позволяет цепочку вызовов: `std::cout « a « b;` или `std::cin » x » y;` .

4.2.6 Какие функции ввода/вывода можно использовать наравне с операторами? Как прочитать целиком строку с пробелами?

Альтернативы/дополнения:

- `std::getline(std::istream&, std::string&)` — читает строку до символа новой строки, включая пробелы.
- Методы потоков: `get()`, `getline()` (член класса), `read()` (для бинарных данных).
- Для форматного ввода/вывода можно использовать `std::scanf`/`printf` (C-style) — но они отличаются по семантике и безопасности.

4.2.7 Что такое состояние потока? Какие флаги идентифицируют состояние (`good`, `eof`, `fail`, `bad`)? Примеры «сломать» поток поправимо и непоправимо; как сбросить флаги?

Состояние потока хранится в битовом наборе флагов:

- `goodbit` — всё в порядке.
- `eofbit` — достигнут конец файла.
- `failbit` — логическая ошибка формата (например, не удалось преобразовать токен).
- `badbit` — серьёзная ошибка (например, аппаратная ошибка/поток сломан).

Примеры:

- Поправимая ошибка: попытка прочитать число из токена, который не является числом ⇒ устанавливается `failbit`, поток можно восстановить с помощью `clear()` и, возможно, удалив неверные данные.
- Непоправимая ошибка: аппаратная ошибка чтения или непоправимый отказ устройства ⇒ `badbit`.

Сброс флагов:

```
if (!std::cin) {
    std::cin.clear();         // сбрасывает флаги
    std::cin.ignore( ... );   // перебросить неверные данные из буфера
}
```

4.2.8 Что такое неявное приведение потока к булевому типу? Как работают проверки `if (s)` и `if (!s)`?

Поток имеет оператор приведения к состоянию (в современном C++ это неявное приведение к булевому контексту через специфику класса потока). Выражение `if (s)` эквивалентно проверке `s.good()` в смысле «поток в хорошем состоянии и готов к операциям». `if (!s)` используется для проверки несостоятельности (например, `fail` или `bad`).

4.2.9 Идиоматические конструкции: чтение файла строка за строкой и чтение слов (токенов)

Чтение строк (включая пробелы):

```
std::string line;
while (std::getline(std::cin, line)) {
    // обработать line
}
```

Чтение слов (токенов, разделённых пробелами):

```
std::string token;
while (std::cin >> token) {
    // обработать token
}
```

Эти конструкции корректно завершаются при достижении EOF или при ошибке потока.

4.2.10 Как переназначить стандартные потоки на другие устройства (например, файлы) без перекомпиляции?

Перенаправление во время запуска:

- В shell: `./program < input.txt > output.txt` (никакой перекомпиляции не требуется).

Переназначение в коде:

```
std::ifstream infile("input.txt");
auto cinbuf = std::cin.rdbuf(infile.rdbuf()); // перенаправляем std::cin
// ... читать из std::cin как обычно ...
std::cin.rdbuf(cinbuf); // восстановить
```

Аналогично для `std::cout` и его `rdbuf()`.

4.2.11 Что такое "кодирование символов"? Как Unicode решает проблему больших наборов символов? Структурное деление и максимальные codepoint'ы.

Кодирование — правило отображения символов (code points) в последовательности байтов. Unicode — это стандарт, задающий уникальное число (code point) для каждого символа; он не фиксирует единственное байтовое представление. Unicode использует модель plane'ов; базовая многоязычная плоскость (BMP) содержит кодовые позиции U+0000..U+FFFF. Полный диапазон Unicode ограничен U+0000..U+10FFFF (то есть 1 114 112 возможных code points).

4.2.12 Пример 8-битной кодировки, ограничения, и что определяет первую половину

Пример: ASCII (7-бит) и расширенные 8-битные кодировки (например, ISO-8859-1). Ограничения: 8-битная кодировка может напрямую представить только 256 различных кодовых позиций; это недостаточно для всех мировых алфавитов. Первая половина 8-битных кодировок (первые 128 кодов) обычно совпадает с ASCII; стандарт ASCII определяет первые 128 символов.

4.2.13 Пример кодировки с переменной длиной (UTF-8), преимущества и недостатки

UTF-8 кодирует Unicode code points в от 1 до 4 байт:

- Преимущества: обратная совместимость с ASCII, эффективная в использовании для латиницы, самосинхронизируемость.
- Недостатки: случайный доступ по «символам» (code points) сложнее, т.к. один символ может занимать несколько байт.

4.2.14 Типы для представления символов в C++ и литералы

- `char` — обычно байт (unit of storage), используют для UTF-8 или байтовых данных.
- `wchar_t` — ширина зависит от реализации; может быть 16 или 32 бита.
- `char16_t` — 16-битный тип для UTF-16 code units.
- `char32_t` — 32-битный тип для UTF-32 code units.

Литералы:

- `u8"..."` — UTF-8 строка (char[] в C++20 пометка менялась, но семантика понятна),
- `u"..."` — char16_t literal,
- `U"..."` — char32_t literal,
- `L"..."` — wide string literal (wchar_t).

4.2.15 Пример статического С-массива и вычисление числа элементов (sizeof); decay массивов

Пример:

```
int arr[] = {1,2,3,4};      // статический массив из 4 элементов
std::size_t n = sizeof(arr) / sizeof(arr[0]); // 4
```

При передаче массива в функцию он *декэйт* (decay) в указатель на первый элемент: формально тип `int[N]` превращается в `int*` в большинстве контекстов (за исключением `sizeof`, `decltype`, инициализации).

4.2.16 Связь между С-массивами и С-строками; можно ли по типу `char*` понять, С-массив это или С-строка?

С-строка — это массив `char` с нуль-терминатором '\0' как признак конца. Тип `char*` сам по себе не указывает, является ли память массивом или строкой; это просто указатель. Для определения того, корректно ли это С-строка, нужна договорённость о содержимом (наличие '\0').

4.2.17 Что такое '\0' и отличие от '0'?

`'\0'` — нулевой символ (ASCII NUL), его код 0; он используется как признак конца С-строки. Символ `'0'` — это символ '0' с кодом 48 (в ASCII). Они разные: `'\0'` означает отсутствие символа (терминатор), `'0'` представляет собой символ цифры ноль.

4.2.18 Для чего используется `std::string`? Основные операции, инициализация, внутреннее представление

`std::string` — удобный класс для хранения и работы со строками. Основные операции: `size()/length()`, `empty()`, `operator+`, `append()`, `substr()`, `c_str()`, индексация `operator[]`, итераторы и т.д.

Инициализация:

```
std::string s1 = "hello";
std::string s2("world");
std::string s3 = s1 + " " + s2;
```

Внутреннее представление: буфер хранится динамически (в куче), но реализации обычно применяют SSO (short string optimization) — когда короткая строка хранится прямо внутри объекта `std::string` без выделения внешней памяти. Стандарт гарантирует, что `c_str()` возвращает непрерывный массив символов, оканчивающийся нулём (начиная с C++11).

4.2.19 Как прочитать токен и как прочитать строку с пробелами в `std::string`?

```
std::string token;
std::cin >> token;           // читает до первого пробельного символа

std::string line;
std::getline(std::cin, line); // читает строку целиком (включая пробелы)
```

4.2.20 Связь между `sizeof(std::string)` и `s.length()`; что такое SSO

`sizeof(std::string)` возвращает размер объекта (фиксированное число байт), зависящее от реализации — это не длина строки. `s.length()` — логическая длина строки в символах. SSO — оптимизация, при которой короткая строка хранится непосредственно в памяти объекта `std::string` (без вложенного выделения динамической памяти), что ускоряет операции для коротких строк.

4.2.21 Преобразования между строками и числами (в стиле C и C++)

- C-style: `std::atoi`, `std::strtol`, `sprintf`.
- C++-style: `std::stoi`, `std::stol`, `std::to_string`.
- Потоковый подход: `std::istringstream` / `std::ostringstream` (или `std::stringstream`).

4.2.22 Как осуществляется конкатенация строк; что эффективнее

- Простейший способ: `a + b` (удобно, но может создавать временные объекты).
- Более эффективно: использовать `a.append(b)` или `reserve()` заранее, чтобы избежать многократных аллокаций:

  ```
  std::string s;
  s.reserve(estimated_size);
  for (...) s.append(chunk);
  ```

- Для множественных конкатенаций удобен `std::ostringstream` или `std::string::append` с резервированием.

4.2.23 Как `std::stringstream` помогает при множественной конкатенации и при преобразованиях

- `std::ostringstream` позволяет «потоково» накапливать строку без явного управления буферами:

  ```
  std::ostringstream oss;
  oss << x << ' ' << y << '\n';
  std::string result = oss.str();
  ```

- Для преобразования числа в строку: `oss « number; std::string s = oss.str();`
- Для преобразования строки в число: `std::istringstream iss(s); iss » number;`

4.2.24 Как `std::stringstream` используется в модульном тестировании?

- Вместо реального файла можно передать `std::istringstream` в функцию, принимающую `std::istream&`, и проверить поведение парсинга:

  ```
  void parse(std::istream &in);

  std::istringstream fake("1 2 3\n");
  parse(fake); // проверяем поведение без файлов
  ```

- Аналогично для вывода: использовать `std::ostringstream` и затем проверить строку, полученную через `oss.str()`.

4.2.25 Какой статус `std::stringstream` в иерархии потоков STL?

`std::stringstream` — специализация потокового класса, комбинирующая вход и выход в памяти; функционально это наследник `std::iostream`, предоставляющий буфер `std::stringbuf`.

Краткая сводка: все контрольные вопросы по теме «Потоки и строки STL» раскрыты:

- стандартные потоки, заголовки, объекты — объяснены;
- иерархия потоков и проблема ромбовидного наследования обсуждены;
- операции `»` / `«`, `getline`, состояние потока и методы управления – приведены;
- идиоматическое чтение строк и токенов показано;
- перенаправление потоков в рантайме — показано;
- кодировки, Unicode, UTF-8, типы символов и литералы — описаны;
- C-массивы, decay, C-строки, '\0' — ясно объяснены;
- `std::string`, .c_str(), SSO, sizeof vs length — разъяснено;
- конкатенация, преобразования и использование `std::stringstream` и тестирование — разобраны.

5 Функциональная декомпозиция, ссылки и константность

5.1 Темы

Материал этого раздела преимущественно рассматривается в рамках лекции 5. Включает:

- Понятие декомпозиции задач и функции как основной строительный блок программы.
- Функциональная декомпозиция в C++: свободные функции и методы, интерфейс/реализация.
- `void` как неполный тип; `void*`.
- Определение функции: заголовок, сигнатура, формальные параметры, тело, `return`.
- Caller / callee / prototype.
- Признаки хорошей функции, smell-коды, принципы декомпозиции.
- Принципы S.O.L.I.D. — кратко с примерами.
- Параметры функций: by-value, by-reference, const-ref, ссылки-результаты, правило «Правило №5».
- CV-квалификаторы: `const`, `volatile`.
- Ссылки: синтаксис, binding, нельзя перепривязать, идиомы (range-based for).
- Идиоматические правила передачи аргументов (мелкие/большие объекты).

5.2 Контрольные вопросы и ответы

5.2.1 Как решить большую задачу так, чтобы решение было корректным, проверяемым, воспроизводимым, переиспользуемым, читаемым?

Основные практики:

- Разбить задачу на маленькие независимые функции (функциональная декомпозиция).
- Для каждой функции формализовать интерфейс (что принимает, что возвращает, побочные эффекты).
- Писать юнит-тесты для каждой функции (изолированное тестирование).
- Использовать инкапсуляцию и модули для управления зависимостями.
- Документировать предусловия/постусловия/инварианты.
- Следовать SOLID-подходам и чистым методикам разработки (одна ответственность, малые модули).

5.2.2 Что такое функция с точки зрения языка C++?

Функция — именованный блок инструкций, принимающий параметры и возвращающий (возможно) значение. Формально состоит из заголовка (имя, список формальных параметров, возвращаемый тип), опциональных спецификаторов и тела (блок инструкций). Функция реализует отношение «вход—выход» и может иметь побочные эффекты.

5.2.3 Как определить функцию / процедура? Какие основные компоненты функции?

Определение включает:

- возвращаемый тип (может быть `void`),
- имя функции,
- список формальных параметров `(type name, ...)` — может быть пуст,
- (опционально) квалификаторы (`const`, `noexcept`, `constexpr`, `inline`, спецификаторы видимости и т.д.),
- тело — блок `{ ... }` с инструкциями,
- инструкция `return` для возврата значения (если тип не `void`).

Пример:

```
int sum(int a, int b) {
    return a + b;
}
```

5.2.4 Что такое неполный тип `void`? Можно ли породить объект такого типа? А типа `void*`?

- `void` — неполный тип, означающий «нет значения»; нельзя создавать объект типа `void` (неопределённая память/безразмерный тип).
- `void*` — указатель на неизвестный тип; можно создавать, хранить и преобразовывать (с приведениями) указатели `void*`. `void*` полезен для хранения "сырых"адресов, но операция разыменования невозможна без приведения к конкретному типу.

5.2.5 Что такое свободная функция и что такое функция-член? Примеры.

- **Свободная (non-member) функция** — функция, не являющаяся частью класса/структуры: `int add(int a, int b);` . Хороша для операций, не требующих доступа к внутреннему состоянию объекта.
- **Функция-член (member)** — определена внутри класса: `struct S { int f(); };`. Имеет неявный параметр `this`.

Пример `std::getline` — свободная функция в пространстве имён `std` (она принимает `std::istream&` в качестве первого аргумента).

5.2.6 Является ли `std::getline` свободной или членом?

`std::getline` — свободная (non-member) функция: она определена как перегруженная функция в `namespace std` и принимает `std::istream&` и `std::string&`.

5.2.7 Является ли `static` функция, объявленная в классе, свободной? Как вызвать?

Статическая функция-член класса `static bar()` всё равно является *функцией-членом* (belongs to class scope) но не имеет неявного `this`. Вызывать можно как `Foo::bar();` или через объект `f.bar()` (но предпочтительна нотация с классом).

5.2.8 Что такое side-effect у функции и почему это нежелательно для свободных функций?

Side effect — изменение состояния вне локальной области видимости функции (изменение глобальных переменных, I/O, модификация параметров через ссылки). Для свободных функций side effects уменьшают переиспользуемость и тестируемость. Хорошая свободная функция должна быть по возможности без побочных эффектов (функция должна быть детерминированной и позволять композицию).

5.2.9 Что такое методы для классов и пример метода `std::string`?

Метод — функция, объявленная внутри класса; имеет доступ к `this` и к членам класса. Пример: `std::string::length()` — метод, возвращающий длину строки.

5.2.10 Что такое список формальных параметров функции? Можно ли составить список только из типов?

Список формальных параметров — это список пар «тип имя» или только типов (имя можно опустить в объявлении). В объявлении/прототипе можно писать только типы:

```
int f(int, double); // допустимо - имена параметров опущены
```

Но в определении обычно указывают имена для использования внутри тела.

5.2.11 Что такое сигнатура функции? Зачем её отделять от заголовка и прототипа?

Сигнатура — набор информации, по которой компилятор различает функции для перегрузки: имя функции + количество и типы (и cv/ref- квалификаторы для методов) формальных параметров. Возвращаемый тип *не* входит в сигнатуру для перегрузки функций. Отдельно рассматривают заголовок/прототип, поскольку сигнатура — это часть интерфейса, а заголовок может содержать также квалификаторы (`noexcept`, `constexpr`, атрибуты) и документацию.

5.2.12 Чем различаются заголовок функции и прототип? Что относится к интерфейсной части?

- **Прототип** (declaration) — объявление функции (ее интерфейса): тип возвращаемого значения, имя, параметры; не содержит тела.
- **Заголовок** — чаще используются как синоним прототипа; но в широком смысле заголовок функции — верхняя строка определения вместе с её спецификаторами.

- Интерфейсная часть = прототип + документация предусловий/постусловий.

5.2.13 Может ли функция иметь более одной точки выхода? Хороший ли это стиль? Может ли `void` функция иметь `return`?

- Функция может иметь несколько инструкций `return`. Это допустимо; стиль — предмет соглашений. В большинстве стилей наличие нескольких точек выхода упрощает ранний выход при ошибках (guard clauses), но может усложнить анализ состояния при cleanup — в C++ RAII решает проблему освобождения ресурсов.
- `void` функция может иметь `return;` без выражения — для раннего выхода.

5.2.14 Нужно ли компилятору полное определение функции? А линкеру?

- Для компиляции вызова функции обычно достаточно прототипа (описания) — компилятору нужно знать сигнатуру, чтобы корректно сгенерировать код вызова.
- Линкер требует полного определения (definition) функции, если функция действительно вызывается и не является inline/templated/constexpr с видимой реализацией в момент компоновки.

5.2.15 Как обычно распределяется интерфейс и реализация по заголовкам и модулем трансляции?

Интерфейсы объявляются в заголовочных файлах (`.h`/`.hpp`) или в модульных интерфейсных единицах; реализации — в `.cpp` или модульных реализациях. Клиенты включают заголовки или импортируют модули, а линковка обеспечивает объединение реализаций.

5.2.16 Признаки хорошей функции (и объяснение)

Признаки:

- **Единая ответственность** — делает только одно логическое действие.
- **Короткая** — небольшое количество строк; легко осмысляется целиком.
- **Понятный интерфейс** — минимальное количество аргументов; аргументы понятны по именам/типам.
- **Отсутствие неожиданных побочных эффектов** — побочные эффекты документированы и минимальны.
- **Реентерабельность / чистота** — если возможно, не использовать глобальное состояние.
- **Хорошая тестируемость** — можно протестировать в изоляции.

5.2.17 Симптомы плохой функции (smell code)

- Слишком длинная функция (много строк, много уровней вложенности).
- Слишком много параметров (>3–4).
- Множество побочных эффектов и скрытых зависимостей.
- Дублирование кода внутри функции.
- Сильная связанность с внешними модулями (затрудняет тестирование).

5.2.18 Что такое реентерабельная функция? Пример параметризуемой и непараметризуемой функции

- **Реентерабельность** — функция может быть прервана и затем повторно вызвана параллельно/последовательно без нарушения общей корректности (не использует модульное глобальное изменяемое состояние).
- **Параметризуемость** — функция имеет параметры, которые определяют поведение. Пример непараметризуемой хорошей функции: `int digits_in_decimal(int n);` — простая, конкретная задача, легко тестируется.

5.2.19 Принцип высокой/низкой связности (coupling & cohesion)

- **Когезия (cohesion)** — степень того, насколько элементы модуля/функции логически связаны. Высокая когезия желательна.
- **Связность (coupling)** — степень зависимости между модулями. Низкая связность желательна.

5.2.20 Тестируемость функции

Тестируемая функция:

- небольшой интерфейс,
- детерминированна (или поведение тестируемо при контролируемых входах),
- не зависит от глобальных состояний (или позволяет их подменять).

5.2.21 Оптимальный размер функции и ясность

Оптимальный размер — такой, чтобы функция выполняла единичную логическую задачу и помещалась в поле зрения разработчика (один экран/несколько экранов). Слишком маленькие функции приводят к росту числа абстракций; слишком большие — к потере ясности.

5.2.22 Принципы функциональной декомпозиции

- Делить проблему на подзадачи с минимальными взаимными зависимостями.
- Выделять интерфейсы и инварианты на границах модулей.
- Делать функции выразительными: имя и сигнатура должны объяснять призвание функции.

5.2.23 Принципы S.O.L.I.D. и примеры

- **S (Single Responsibility)**: класс/функция — одна причина для изменения. Пример: класс `FileReader` только читает файлы; логика парсинга вынесена в отдельный класс.
- **O (Open/Closed)**: сущности открыты для расширения, закрыты для модификации. Пример: использование виртуального интерфейса для добавления поведения без изменения существующего кода.

- **L (Liskov Substitution)**: подкласс должен быть взаимозаменяем с базовым классом. Пример: подкласс не должен усиливать предусловия и не должен ослаблять постусловия.
- **I (Interface Segregation)**: мелкие специализированные интерфейсы лучше, чем один большой. Пример: разделение интерфейса `IPrinter` на `ITextPrinter` и `IImagePrinter`.
- **D (Dependency Inversion)**: зависеть от абстракций, а не от конкретных реализаций. Пример: передавать `ILogger&` в конструкторе вместо использования конкретного `ConsoleLogger`.

5.2.24 Чем отличаются фактические параметры (arguments) от формальных параметров?

- **Формальные параметры** — имена/типы, объявленные в заголовке функции.
- **Фактические параметры (arguments)** — значения/выражения, передаваемые при вызове.

5.2.25 Что такое перегрузка функций и условия для перегрузок?

Перегрузка — наличие нескольких функций с одним именем, различающихся сигнатурами (количество/типы параметров, cv/ref- квалификаторы для методов). Возвращаемый тип не отличает перегрузки. Пример:

```
void f(int);
void f(double);
```

5.2.26 Методы графического представления взаимодействия функций

- **Call graph** (граф вызовов) — вершины = функции, ребра = вызовы.
- **Sequence diagram (UML)** — временная диаграмма взаимодействия объектов/функций.
- **Activity diagram** — поток управления.

5.2.27 Что такое CV-квалификаторы? Что такое `const` и `volatile`?

- **CV-квалификаторы** — `const` и `volatile` (часто вместе с `restrict` в других языках).
- `const` запрещает изменение объекта через данную ссылку/имя; влияет на семантику операций (методы `const`).
- `volatile` указывает компилятору, что объект может изменяться внешне (аппаратно или потоками) — компилятор должен избегать оптимизаций, предполагающих неизменность.
- `const` используется гораздо чаще; `volatile` редко (низкоуровневое программирование, драйверы).

5.2.28 Что такое константный объект (const)? Есть ли у него имя, адрес, тип, значение? Что меняет `const`?

Константный объект имеет имя, адрес (если lvalue), тип и значение. `const` запрещает операции, меняющие значение через этот идентификатор; не влияет на представление объекта в памяти, но влияет на операции, доступные в языке (компилятор контролирует модификации).

5.2.29 Что такое ссылка на объект и зачем она нужна? Как задаётся?

Ссылка — альтернативное имя для существующего объекта. Синтаксис: `T& r = obj;` . Ссылка не хранит адрес как значение в обычном понимании, и её нельзя перепривязать. Ссылки используются для:

- передачи больших объектов без копирования (`const T&`),
- изменения объекта в вызывающем коде (неконстантная ссылка),
- полиморфизма (передача базового по ссылке).

Пример binding:

```
int x = 42;
int &r = x; // r и x - одно и то же имя (alias)
```

5.2.30 Почему говорят, что ссылка — «еще одно имя для объекта»? Примеры.

Если у объекта есть несколько имён (переменная, ссылка, указатель, элемент контейнера), они ссылаются на одну и ту же область памяти. Примеры:

- `int x; int &r = x;` — изменение через `r` изменяет `x`.
- range-based for с ссылкой: `for (auto &e : vec) e += 1;` — меняет элементы в контейнере.

5.2.31 Можно ли перепривязать ссылку? Отложенное привязывание?

- **Перепривязать ссылку нельзя.** Ссылка привязывается к объекту при инициализации и далее остаётся с этим объектом (под капотом реализовано через указатель, но семантически — неизменяемое alias).
- **Отложенное привязывание** не поддерживается для обычных ссылок. Альтернативы: указатели (`T*`) или `std::reference_wrapper<T>` / `std::optional<std::reference_wrapper<`

5.2.32 Когда параметры передаются по ссылке? Чем отличаются ссылки от передачи по значению?

- **By value** (по значению) — копируется объект (безопаснее, независимее).
- **By reference** (по ссылке) — передаётся alias; нет копирования; callee может изменить объект в caller'е.
- Используют ссылки, чтобы избежать копирования больших объектов или чтобы позволить функции менять аргумент.

5.2.33 Как вернуть из функции более одного значения (не используя массивы/структуры)?

Варианты:

- Использовать выходные параметры по ссылке/указателю:

  ```
  void f(int input, int &out1, double &out2) { ... }
  ```

- Возвращать `std::pair` / `std::tuple` / пользовательскую структуру.
- В C++17 можно использовать structured bindings: `auto [a,b] = f();`.

5.2.34 Что такое «Правило № 5»: «В C++ по умолчанию — всё по значению»?

Это практическое наблюдение: при отсутствии явного `&` или указателя аргументы и возвраты копируются по значению. Чтобы избежать ненужных копий, используют:

- `const T&` для передачи больших объектов на чтение,
- `T&` для модификации,
- `T&&` и `std::move` для перемещения/может оптимизировать копии.

5.2.35 Что такое константная ссылка и когда её использовать?

`const T&` — ссылка, которая не позволяет изменять объект. Используют когда нужно передать большой объект для чтения без копирования:

```
void print(const std::string &s) { std::cout << s; }
```

Если объект мал (например, `int`, `double`), копирование дешевле и предпочтительнее.

5.2.36 Пример, где константная ссылка эффективна; когда предпочесть копирование

- Эффективна: `void process(const std::vector<std::string>& v);` — избегает копирования большого вектора.
- Копирование предпочтительнее: `void inc(int x);` — копирование `int` дешевле, чем использование ссылки.

5.2.37 Правило трёх указателей (ПТУ) — «большие» и «малые» объекты

Неформальная эвристика: если тип по размерам больше примерно 2–3 указателей (`sizeof(T) > 3 * sizeof(void*)`), считать его «большим» и передавать по `const T&`. Для типов меньшего размера (скалярные) передавать по значению.

5.2.38 Примеры: когда передавать по значению / по неконстантной ссылке / по константной ссылке

- По значению: `int`, `double`, небольшие POD-структуры.
- По неконстантной ссылке: `void update(Point &p);` — когда нужно изменить аргумент и вернуть изменения в caller.
- По константной ссылке: `void print(const std::string &s);` — большой объект, только чтение.

5.3 Короткий справочник-резюме (чтобы ничего не забыть)

- Прототип в заголовке, определение в `.cpp` (или экспортируемый модуль).
- Компилятор нужен только интерфейс; линкер — реализация.
- Используй `const T&` для больших объектов на чтение; `T` для малых; `T&` для изменения.
- Ссылки нельзя перепривязать; указатели можно.
- `void` — неполный тип; `void*` хранит адрес.
- SOLID + маленькие функции + низкая связность = легко тестируемый код.